

Corso di Laboratorio di Programmazione per Sistemi Mobile e Tablet

Docente: Dr. Mauro Dragoni

Dipartimento di Ingegneria e Scienze dell'Informazione
Anno Accademico 2025/2026

| in questa lezione...

- Android File System
- Internal Storage
- External Storage
- SQLite Database

Salvare i dati

- Shared Preferences – Salvataggio privato di dati (tipi primitivi) in coppie chiave/valore.
- Internal Storage – Salvataggio privato di dati sulla memoria del dispositivo.
- External Storage – Salvataggio pubblico di dati sul device o su una memoria esterna.
- SQLite Databases – Salvataggio di dati strutturati in un database privato.
- Content Providers – Salvataggio privato di dati resi poi accessibili pubblicamente.

Salvare i dati al di fuori di Android

- Network Connection – Salvataggio sul web tramite il proprio server.
- Cloud Backup – Backup dei dati utenti e dei dati dell'applicazione all'interno del cloud.
- Firebase Realtime Database – Salvataggio e sincronizzazione dei dati con database NoSQL presenti sul cloud.

Android File System

- External storage: directories pubbliche.
- Internal storage: directories private per la propria applicazione.
- Le applicazioni possono navigare attraverso la struttura delle directories.
- Operazioni e strutture dati simili a cio' che normalmente avviene con la libreria java.io su Linux.

Internal storage

- Sempre disponibile.
- Utilizza il file system del device.
- Solo la propria applicazione puo' accedere a quei file (a meno di essere specificati in modo esplicito come di sola lettura e/o scrittura).
- All'atto di rimozione dell'applicazione, il sistema rimuove dall'internal storage tutti i files creati dall'applicazione stessa.

External storage

- Non sempre disponibile in quanto puo' essere rimossa.
- Utilizza il file system del device corrente o memoria esterna fisica (es. SD card).
- Lettura pubblica, quindi qualsiasi altra applicazione puo' accedere ai dati generati.
- All'atto di rimozione dell'applicazione, il sistema non rimuove i file generati dalla vostra applicazione.

Quando utilizzare la memoria interna o esterna

La memoria interna e' consigliata quando:

- si vuole essere certi che nessun altro utente o applicazione sia in grado di accedere ai propri files.

La memoria esterna e' consigliata quando i files:

- non necessitano di restrizioni di accesso;
- si vogliono condividere con altre applicazioni;
- si vogliono rendere accessibili tramite altri dispositivi (es. computer).

Internal Storage

- Utilizza directories private dedicate alla vostra applicazione.
- L'applicazione ha sempre i permessi di lettura e scrittura.
- Per accedere alle cartelle presenti nella memoria “permanente”: **filesDir**
(getFilesDir() in Java)
- Per accedere alle cartelle presenti nella memoria “temporanea”: **cacheDir**
(getCacheDir() in Java)

Scrittura/Lettura file

```
// scrittura
val file = File(filesDir, "notes.txt")
file.writeText("This is my note")

// lettura
val content = File(filesDir, "notes.txt").readText()
```

External Storage

- Salvataggio su device o SD card
- Impostare i permessi all'interno del manifesto
 - i permessi di scrittura includono quelli di lettura.

```
<uses-permission  
    android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
```

```
<uses-permission  
    android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

Controllare sempre la disponibilit  della memoria esterna

```
fun isExternalStorageAvailable(): Boolean {  
    return Environment.getExternalStorageState() == Environment.MEDIA_MOUNTED  
}
```

```
fun isExternalStorageReadable(): Boolean {  
    val state = Environment.getExternalStorageState()  
    return state == Environment.MEDIA_MOUNTED || state ==  
Environment.MEDIA_MOUNTED_READ_ONLY  
}
```

```
fun isExternalStorageWritable(): Boolean {  
    return Environment.getExternalStorageState() == Environment.MEDIA_MOUNTED  
}
```


Controllare sempre la disponibilit  della memoria esterna

```
fun saveToExternalStorage(context: Context, filename: String, content: String) {  
    if (isExternalStorageWritable()) {  
        val file = File(context.getExternalFilesDir(null), filename)  
        file.writeText(content)  
    } else {  
        Log.e("Storage", "External storage is not available for writing")  
    }  
}
```

```
fun readFromExternalStorage(context: Context, filename: String): String? {  
    if (isExternalStorageReadable()) {  
        val file = File(context.getExternalFilesDir(null), filename)  
        if (file.exists()) {  
            return file.readText()  
        }  
    }  
    return null  
}
```

Alcuni esempi di directories pubbliche esterne

- DIRECTORY ALARMS e DIRECTORY RINGTONES
Per accedere a file audio da usare per allarmi o suonerie.
- DIRECTORY DOCUMENTS
Per accedere i documenti creati dall'utente.
- DIRECTORY DOWNLOADS
Per accedere ai file scaricati dall'utente.

developer.android.com/reference/android/os/Environment.html

Accesso a cartelle esterne pubbliche - MediaStore

```
fun saveImageToGallery(context: Context, bitmap: Bitmap, displayName: String) {  
    val contentValues = ContentValues().apply {  
        put(MediaStore.Images.Media.DISPLAY_NAME, displayName)  
        put(MediaStore.Images.Media.MIME_TYPE, "image/png")  
        put(MediaStore.Images.Media.RELATIVE_PATH, Environment.DIRECTORY_PICTURES + "/MyApp")  
    }  
  
    val resolver = context.contentResolver  
    val uri = resolver.insert(MediaStore.Images.Media.EXTERNAL_CONTENT_URI, contentValues)  
  
    uri?.let {  
        resolver.openOutputStream(it)?.use { stream ->  
            bitmap.compress(Bitmap.CompressFormat.PNG, 100, stream)  
        }  
    }  
}
```

Accesso a cartelle esterne pubbliche – Storage Access

```
val createDocument = registerForActivityResult(
    ActivityResultContracts.CreateDocument("text/plain")
) { uri ->
    uri?.let {
        contentResolver.openOutputStream(it)?.use { stream ->
            stream.write("File content here".toByteArray())
        }
    }
}

// Launch the file picker
createDocument.launch("report.txt")

private fun writeToUri(uri: Uri, content: String) {
    try {
        contentResolver.openOutputStream(uri)?.use { stream ->
            stream.write(content.toByteArray())
        }
        Log.d("SAF", "File written successfully")
    } catch (e: IOException) {
        Log.e("SAF", "Error writing file", e)
    }
}
```


Quanto spazio e' rimasto?

- Se non ci fosse abbastanza spazio, viene lanciata un'eccezione di tipo [IOException](#)
- Se si conosce la dimensione del file da salvare, controllare lo spazio rimasto tramite:
 - [getFreeSpace\(\)](#)
 - [getTotalSpace\(\)](#)
- Se non si conosce la dimensione del file utilizzare il blocco try/catch.

Eliminare i files non piu' necessari

- External storage: `myFile.delete()`
- Internal storage: `deleteFile(fileName)`

Come evitare di cancellare i files utente

Quando un utente disinstalla l'applicazione, tutto il contenuto dell'area privata viene eliminato.

Non utilizzare l'area privata per salvare i contenuti che appartengono all'utente!

Ad esempio:

- Foto catturate o modificate tramite l'applicazione.
- File musicali che l'utente ha comprato tramite l'applicazione.

SQLite Database

- Ideale per salvare strutture dati ripetute (es. contatti).
- Android fornisce un meccanismo di gestione di database SQL-like da poter utilizzare.
- Passi:
 - creare la struttura del database: preparare uno script SQL che crei la struttura interna del database;
 - creare una classe che estenda SQLiteOpenHelper: questa classe servirà a gestire la nascita e l'aggiornamento del database su memoria fisica e a recuperare un riferimento all'oggetto SQLiteDatabase, usato come accesso ai dati;
 - creare una classe per l'interazione con il database: solitamente questa ha due caratteristiche: (1) contiene un riferimento all'oggetto helper definito al punto precedente, (2) contiene i metodi con cui, dalle altre componenti dell'applicazione, verranno richieste operazioni e selezioni sui dati.

SQLite Database - Esempio

```
public class DBhelper extends SQLiteOpenHelper
{
    public static final String DBNAME="BILLBOOK";
    public DBhelper(Context context) {
        super(context, DBNAME, null, 1);
    }

    @Override
    public void onCreate(SQLiteDatabase db)
    {
        String q = "CREATE TABLE "+DatabaseStrings.TBL_NAME+
            " ( _id INTEGER PRIMARY KEY AUTOINCREMENT," +
            DatabaseStrings.FIELD_SUBJECT+" TEXT," +
            DatabaseStrings.FIELD_TEXT+" TEXT," +
            DatabaseStrings.FIELD_DATE+" TEXT) ";
        db.execSQL(q);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion)
    { }
}
```

SQLite Database – gestione del metodo onUpgrade

- Le problematiche relative alla gestione delle diverse version di un database le abbiamo anche qui!
- Il metodo **onUpgrade** viene invocato automaticamente quando la versione del database presente sullo smartphone e' obsoleta rispetto a quella definita nel codice.
- Come scrivere il codice in modo efficace?

SQLite Database – gestione del metodo onUpgrade

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion)
{
    db.execSQL(ALTER_USER_TABLE_ADD_USER_SOCIETY);
    db.execSQL(ALTER_USER_TABLE_ADD_USER_STREET1);
}
```

SQLite Database – gestione del metodo onUpgrade

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion)
{
    db.execSQL(ALTER_USER_TABLE_ADD_USER_SOCIETY);
    db.execSQL(ALTER_USER_TABLE_ADD_USER_STREET1);
}
```

- NO

Problema: queste istruzioni verranno eseguite ogni volta che si aggiorna l'app. Se si passa dalla versione 1 alla 2, verranno eseguite e verranno aggiunte le colonne. Quindi, se aggiorni da 2 a 3 (e non si modifica questo codice), si avrà un errore quando si tenta di aggiungere nuovamente queste colonne. Si potrebbe essere tentati di modificare questa sezione ogni volta che si cambia l'app, ma questo è soggetto a errori in quanto non tutti gli utenti potrebbero avere la versione corrente installata.

SQLite Database – gestione del metodo onUpgrade

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {

    // If you need to add a column
    if (newVersion > oldVersion) {
        db.execSQL("ALTER TABLE foo ADD COLUMN new_column INTEGER DEFAULT 0");
    }
}
```

SQLite Database – gestione del metodo onUpgrade

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {

    // If you need to add a column
    if (newVersion > oldVersion) {
        db.execSQL("ALTER TABLE foo ADD COLUMN new_column INTEGER DEFAULT 0");
    }
}
```

- NO

Problema: si potrebbe dare a questa opzione il beneficio del dubbio. Il problema è che ogni volta che si verifica un aggiornamento, l'istruzione IF è vera e ci rimane lo stesso identico problema del frammento di codice precedente.

SQLite Database – gestione del metodo onUpgrade

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    String upgradeQuery = "ALTER TABLE mytable ADD COLUMN mycolumn TEXT";
    if (oldVersion == 1 && newVersion == 2)
        db.execSQL(upgradeQuery);
}
```

SQLite Database – gestione del metodo onUpgrade

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    String upgradeQuery = "ALTER TABLE mytable ADD COLUMN mycolumn TEXT";
    if (oldVersion == 1 && newVersion == 2)
        db.execSQL(upgradeQuery);
}
```

- **NO**

Problema: questo è un po' più difficile da individuare. Si consideri cosa succede se un utente passa dalla versione 1 alla 3. Mancherà completamente la query di aggiornamento. Questi utenti si troveranno in un limbo in cui mancano alcuni degli aggiornamenti intermedi e non avranno lo schema SQL previsto.

SQLite Database – gestione del metodo onUpgrade

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    if (oldVersion < 2) {
        db.execSQL(DATABASE_ALTER_TEAM_1);
    }
    if (oldVersion < 3) {
        db.execSQL(DATABASE_ALTER_TEAM_2);
    }
}
```

SQLite Database – gestione del metodo onUpgrade

```
@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
    if (oldVersion < 2) {
        db.execSQL(DATABASE_ALTER_TEAM_1);
    }
    if (oldVersion < 3) {
        db.execSQL(DATABASE_ALTER_TEAM_2);
    }
}
```

- SI

Approfondimenti

Files:

- [Saving Files](#)
- [getExternalFilesDir\(\) documentation and code samples](#)
- [getExternalStoragePublicDirectory\(\) documentation and code samples](#)
- [java.io.File class](#)
- [Oracle's Java I/O Tutorial](#)

[Backups:](#)

- [Configuring Auto Backup for Apps](#)
- [Using the Backup API](#)
- [Data Backup](#)